AI Pipelines for Document Analysis — From Text to Structured Insight

Note to the Reader

> This chapter introduces **multi-stage AI pipelines** — the pattern behind any application that takes a document, extracts meaning from it, and produces structured, actionable feedback. It is intentionally thorough, because the concepts here are unfamiliar territory even for developers who have already built chatbots.
>
> If you're already comfortable with LLM completion calls, the `messages` array, and basic prompt design, you can skim §1 and §2 and go deeper from §3 onwards. If any of that feels new, work through this chapter top-to-bottom — every section builds on the one before.
>
> The companion tutorial will ask you to *apply* these ideas; this chapter is where they are *explained*. You do not need to write any code now. Your only job is to build the mental model.

Learning Outcomes

After studying this chapter, you will be able to:

- Explain why a multi-stage AI pipeline produces better results than a single large prompt.
- Define unstructured data, structured output, extraction pipeline, evaluation pipeline, and feedback-only design.
- Apply the ICCO framework — to write effective extraction and evaluation prompts.
- Describe what happens inside a text-extraction step before an LLM ever sees a document.
- Distinguish between an AI task that *generates* content and one that *evaluates* existing content, and explain why this distinction matters for trust.
- Identify at least four ways that LLM-produced JSON can be malformed, and describe how to handle each defensively.
- Apply a weighted scoring formula to aggregate sub-scores into a single numeric rating.

Prerequisites

- LiteLLM completion calls, the `messages` array, `json.loads` / `json.dumps`, `try/except` with retry, `.env` secrets management, ICCO prompt design.

> 📖 **Optional reading:** Any general guide to ATS-friendly résumé writing and keyword optimisation is a useful real-world companion once you reach §6, though nothing here requires one.

> 🎯
>
> **What You're Preparing For**
>
> In the upcoming tutorial you'll work with an application that accepts two inputs — a résumé PDF and a job description — and produces a structured analysis: a relevancy score and a set of actionable feedback items. This chapter builds the conceptual foundations you need to understand *how* and *why* such a system works. You won't write any code yet; your job here is to build the mental model so that the implementation feels natural when you reach it.

---

§1 Motivation

Every chatbot you have built so far operates the same way: a human sends a message, the model replies, and the conversation continues. This works beautifully when the goal is *conversation*. It is surprisingly inadequate when the goal is *analysis*.

Consider what happens when a recruiter needs to evaluate two hundred job applications in a single afternoon. Each application is a document — a resume, a cover letter, a portfolio link. To do the job well, the recruiter must compare each document against a checklist: the required skills, the right experience level, the expected seniority. The problem is not that the information isn't there. The problem is that it is buried inside free-form prose, formatted inconsistently, and scattered across sections with different names.

Now imagine asking a single chatbot to handle all of this: *"Read this resume, decide if the person has the skills listed in this job posting, give me a percentage match, and explain what's missing — all in one response."* The chatbot will do *something*, but that something will often be a narrative paragraph that's hard to compare against other applicants, may hallucinate skills the applicant didn't list, and almost certainly won't produce the same format twice in a row.

The solution is the same one software engineers use whenever a single function is asked to do too many different things: **break the problem apart**. Give each subtask its own prompt, its own context, and its own output format. Build a pipeline where the output of one stage becomes the structured input to the next. That is the core idea of this chapter — and it is the idea behind every serious AI-assisted document analysis system you will encounter in practice.

> 💡
>
> **The key insight:**
>
> A chatbot is one generalised model doing many things acceptably. A pipeline is many specialised prompts, each doing one thing reliably. Reliability matters when decisions affect people.

---

§2 Big Picture

§2.1 The Assembly-Line Analogy

Think of a document analysis pipeline the way a product designer thinks about a manufacturing assembly line. In an assembly line, raw material enters one end and a finished product comes out the other. Crucially, *each station on the line does exactly one job*. The welding station doesn't paint. The painting station doesn't bolt parts together. No station tries to do everything.

A document analysis pipeline works the same way:

- **Station 1 — Parse**: Raw file bytes go in; clean plain text comes out. No AI involved at this stage.
- **Station 2 — Extract**: Plain text goes in; a structured data object (a JSON dictionary) comes out. This is where the LLM first appears — its job is *extraction*, not evaluation.
- **Station 3 — Analyse**: Two structured data objects go in (the document profile and the requirements profile); a comparison report comes out. This is where the LLM evaluates.
- **Station 4 — Score & Report**: Evaluation data goes in; a human-readable, numeric report comes out.

Each station is independent. You can test it in isolation. You can swap it out without touching the others. You can understand exactly what it received and exactly what it produced.

Figure 4.1: A four-stage document analysis pipeline. Each stage has a single responsibility: parsing produces clean text; extraction turns text into a typed JSON object; analysis compares two JSON objects; scoring produces a numeric report. Data flows left to right; no stage looks back.

§2.2 The Data Transformations in One View

Before going deeper, it helps to see what the data actually looks like at each stage boundary. The transformation is dramatic.

Stage boundary	Example "before"	Example "after"
File → Text	`resume.pdf` (binary bytes)	`"Aiko Tanaka\nSoftware Engineer\nSkills: Python, AWS…"`
Text → JSON	`"…proficient in Python, worked on cloud projects…"`	`{"skills": {"languages": ["Python"], "tools": ["AWS"]}}`
JSON → Analysis	`{"skills": …}` + `{"required_skills": ["Python", "Docker"]}`	`{"present": ["Python"], "missing": ["Docker"], "score": 72}`
Analysis → Report	`{"score": 72, "missing": ["Docker"]}`	`"ATS score: 72/100. Missing keyword: Docker. Action: Add a project that uses containerisation."`

Every transformation involves *reducing ambiguity*. Free-form prose is maximally ambiguous — it says a lot, but a computer can't reliably act on it. A JSON object is maximally unambiguous — every field has a name, a type, and a predictable location. The pipeline's job is to move from the first toward the second, one stage at a time.

---

§3 Data Shapes: Unstructured In, Structured Out

§3.1 What "Unstructured" Really Means

The word unstructured data sounds like a criticism. It isn't. A beautifully written cover letter is unstructured data. A poem is unstructured data. A transcript of a conversation is unstructured data. Unstructured simply means: the information is encoded in natural language, where the *meaning* is clear to a human reader but the *location* of any specific fact is not predictable.

If you want to know whether a cover letter mentions Python, you cannot do this:

```
# This will not work — there is no "skills" field in a text string.
if cover_letter["skills"]["languages"]["Python"]:
    ...
```

You have to either search for the word "Python" (which misses "python" and "py" and "built APIs in Python 3"), or you send the letter to an LLM and ask it to extract the skills. The LLM can read prose the way a human can; a Python `dict` lookup cannot.

Structured data, by contrast, has a predictable shape. A JSON object has named fields. Arrays have typed elements. You can iterate over them, count them, compare them, and store them in a database. Code can act on structured data reliably.

The insight that powers document analysis systems is this: **an LLM is exceptionally good at bridging the gap between unstructured and structured**. It reads prose; it writes dictionaries.

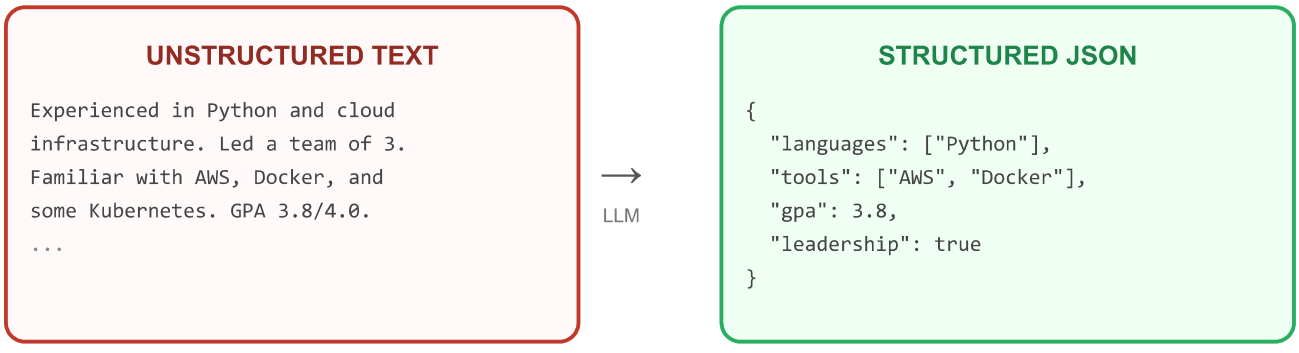

Figure 4.2: The LLM acts as a translator. The same facts that a human reads fluently in prose are re-expressed as a typed, machine-readable JSON object that Python code can interrogate directly.

MULTIPLE CHOICE ch4-q1

A colleague suggests searching a document for the keyword "machine learning" using Python's `str.find()` instead of sending it to an LLM. Which scenario would cause this approach to silently miss a valid match?

- ○ The candidate wrote 'Machine Learning' with capital letters.
- ○ The candidate described building 'predictive models using neural networks' without using the phrase 'machine learning'.
- ○ The document contains the word 'machine' and the word 'learning' on separate lines.

Check

§3.2 JSON as the Universal Bridge

Here we need to make the role of JSON more precise.

In a document analysis pipeline, JSON is not just a convenient output format — it is a **contract**. When Stage 2 (Extraction) produces a JSON object with a field called `"skills"`, Stage 3 (Analysis) is written to expect *exactly* that field. If Stage 2 suddenly started calling it `"skill_set"`, Stage 3 would break silently — `result.get("skills", [])` would return an empty list and no error would be raised.

This means that before you write a single line of prompt text, you should design the JSON schema. What fields does each stage need to receive? What fields should it produce? This discipline — schema-first design — is why you define the output schema before writing a prompt. It is not just a style preference; it is the engineering principle that makes multi-stage pipelines maintainable.

> **Listing 4.1** — A minimal schema definition written as comments before any prompt. This is reference code; your tutorial will ask you to write a version of it.

```
# Schema for Stage 2 output (document profile).
# Stage 3 depends on every field listed here — change with caution.
DOCUMENT_PROFILE_SCHEMA = {
    "name":       "string",          # extracted from header
    "skills":     ["string"],        # normalised skill names
    "experience": ["string"],        # job/project titles
    "education":  "string",          # highest degree
}

# Schema for Stage 3 output (analysis result).
```

```
ANALYSIS_RESULT_SCHEMA = {
    "present":    ["string"],        # requirements found in the document
    "missing":    ["string"],        # requirements not found
    "score":      "integer (0-100)", # percentage of requirements met
    "notes":      "string",          # one-sentence summary
}
```

MULTIPLE CHOICE ch4-q2

You change the field name `"missing"` to `"gaps"` in your Stage 3 prompt output, but forget to update the Stage 4 code that reads the result. What happens when Stage 4 runs?

- ○ A `KeyError` is raised immediately, making the bug obvious.
- ○ Stage 4 silently receives an empty list for `missing`, producing a report that claims no requirements are missing — even when they are.
- ○ Python raises a `JSONDecodeError` because the JSON is now invalid.

Check

§3.3 Schema-First Prompt Design

The ICCO framework maps directly onto schema-first pipeline design. When writing an extraction prompt, the **Output** component of ICCO *is* the JSON schema. You provide the schema in the prompt, and you ask the model to fill it in.

Here is how the four ICCO components translate for an extraction prompt:

⚠️

Common mistake:

Writing the Output component as "a nicely formatted summary" instead of a schema. Summaries are unstructured — they defeat the entire purpose of the extraction stage.

FILL IN THE BLANK ch4-q3

In ICCO terms, the JSON schema you want the model to produce belongs in the ... component.

Check

§4 The Multi-Stage Pipeline

§4.1 Why One Call Cannot Do Everything

A natural first instinct is to write one big prompt: *"Read this document, compare it against these requirements, identify gaps, score it from 0 to 100, list what's missing, and write a one-paragraph feedback summary — all in one response."*

This feels efficient. In practice, it produces worse results than multiple specialised calls, for three concrete reasons:

Reason 1 — Attention dilution. A model attending to five subtasks simultaneously pays less attention to each one. The extraction quality drops. The scoring logic gets tangled with the narrative. The feedback references things it shouldn't have invented.

Reason 2 — Format conflicts. Extraction wants a tightly constrained JSON object. Feedback wants flowing prose. These two output formats are in tension inside a single response. The model will often produce a hybrid that fails cleanly as neither.

Reason 3 — No intermediate checking. In a pipeline, you can inspect and validate the output of Stage 2 before it reaches Stage 3. In a single-call approach, if the extraction was wrong, the analysis is built on a bad foundation and you have no way to detect it.

The principle at stake is: *separation of concerns*. You want separate parts — each with a single responsibility. The same discipline applies to prompt design.

§4.2 Stage Independence

Stage independence means that each stage of the pipeline can be understood, tested, and debugged without knowing anything about the stages around it. To test Stage 2 (Extraction), you give it a sample document and inspect its JSON output. You do not need Stage 3 or Stage 4 to exist yet.

Smoke-test each layer; integrate after each layer is verified.

Design Rule

A well-designed pipeline stage has a clean *input contract* (what it expects to receive) and a clean *output contract* (the JSON schema it promises to produce). If you can describe both in a comment above the function, the stage is properly independent.

MULTIPLE CHOICE ch4-q4

You are debugging a pipeline where Stage 3 (Analysis) is producing incorrect gap reports. Which of the following tests would confirm whether the bug is in Stage 2 (Extraction) or Stage 3 (Analysis)?

- ○ Run the full pipeline end-to-end with a known document and compare the final report to a manually produced one.
- ○ Feed Stage 3 a known-correct Stage 2 output that you wrote by hand, then inspect Stage 3's output.
- ○ Increase `temperature` to 0.9 on Stage 3 and re-run to see if the output changes.

Check

§4.3 Data Flow Between Stages

In a Python pipeline, the output of one stage becomes the input to the next. Because each stage communicates via a Python dictionary (parsed from JSON), the connection is just a function call passing that dictionary:

> **Listing 4.2** — Schematic of data flowing through a two-stage pipeline. Reference only; your tutorial will ask you to fill in the implementations.

```
# Stage 1: parse the file into plain text (no LLM yet)
raw_text = parse_document(file_path)            # str

# Stage 2: extract structured profile (one LLM call)
doc_profile = extract_profile(raw_text)         # dict

# Stage 3: analyse against requirements (one LLM call)
analysis = analyse_against_requirements(
    doc_profile,
    requirements_text                           # also parsed earlier
)                                               # dict

# Stage 4: aggregate and report (no LLM needed)
report = build_report(analysis)                 # str or dict
```

Notice: no stage knows the internals of any other stage. `extract_profile` receives a string and returns a dict. `analyse_against_requirements` receives two dicts and returns a dict. Each can be swapped independently.

TRACE THE FLOW ch4-q5

Arrange these pipeline stages in the correct execution order for a document analysis system.

Call the LLM to compare the two structured profiles

Parse the requirements document into a structured profile

Read the raw bytes from the uploaded file

Build the human-readable feedback report

Call the LLM to extract a structured profile from the document text

Check Order

§5 Document Parsing: Getting Text Out

§5.1 Why Files Are Not Just Text

When you open a PDF in a reader like Adobe Acrobat or Chrome, you see beautifully formatted text. It is tempting to assume that PDF files are just text files with some formatting attached. They are not.

A PDF (Portable Document Format) stores pages as a sequence of *drawing instructions*: "draw glyph A at position (72, 680)", "draw glyph p at position (78, 680)", and so on. The instructions tell the PDF reader how to render each letter on screen. The semantic content — the words, sentences, paragraphs, and sections — must be reconstructed from those low-level drawing operations.

Text extraction is the process of recovering the readable text from those drawing instructions. A library like `pypdf` does this automatically for most PDF files: it reads the drawing instructions, identifies glyph sequences, and concatenates them into a plain text string. But reconstruction is imperfect. The extracted text may have:

- Extra spaces or missing spaces between words (glyphs placed at nearby but not adjacent positions)
- Two-column layouts merged into a single stream of alternating fragments
- Bullet points converted to odd Unicode characters
- Headers and footers interspersed with body text
- No content at all — because the file is a *scanned image* with no text layer

⚠️

Image-based PDFs:

A document scanned to PDF does not contain any text layer. Text extraction returns an empty string. The fix (sending the image to an OCR service) is a topic we won't cover at this time; for now, all documents should be exported from a word processor as a native PDF, not scanned.

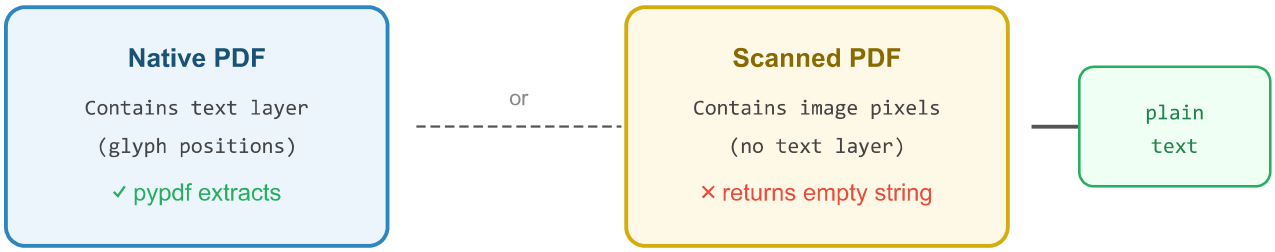

Figure 4.3: A native PDF (exported from a word processor) contains a text layer that extraction libraries can recover. A scanned PDF contains only pixel data; text extraction returns an empty string. Verifying the character count before any LLM call catches this class of failure early.

§5.2 Validating the Extracted Text

Before passing extracted text to an LLM, it is worth verifying that it actually contains meaningful content. Two simple checks cover the most common failure modes:

Minimum length check — If the extracted text is shorter than, say, 200 characters, the extraction almost certainly failed (image PDF, empty document, or a file that is not actually a resume). Raise an informative error before spending a token.

Maximum length check — LLMs have context windows. A very long document (say, 150 pages of scanned academic work) would exceed most models' limits. Warn the user and truncate to a safe size. (One could handle this *properly* with chunking and retrieval; for now, truncation is the pragmatic fallback.)

Listing 4.3 — Minimal input validation after text extraction. Reference only.

```
def validate_extracted_text(text: str, source: str = "document") -> str:
    """Raise ValueError if text is too short; warn and truncate if too long."""
    MIN_CHARS = 200
    MAX_CHARS = 24_000  # ≈ 6,000 tokens at 4 chars/token

    if len(text) < MIN_CHARS:
```

```
        raise ValueError(
            f"{source} text is too short ({len(text)} chars). "
            "The file may be image-based or empty."
        )
    if len(text) > MAX_CHARS:
        print(f"WARNING: {source} text truncated to {MAX_CHARS} chars.")
        return text[:MAX_CHARS]

    return text
```

This pattern — **validate at the boundary before crossing into the LLM layer** — is a general principle for any AI application. Bad input produces bad output; catching it early produces an informative error instead of a confusing AI-generated non-answer.

SPOT THE ERROR ch4-q6

This function is supposed to validate extracted text but has a bug. Which line is wrong, and what should it do instead?

```
1  def validate_text(text):
2      if len(text) > 200:
3          raise ValueError("Text too short!")
4      if len(text) > 24000:
5          text = text[:24000]
6      return text
```

Show Answer

---

§6 Prompt Engineering for Analysis Tasks

Analysis prompts are distinct. This section explains what makes them different.

§6.1 Extraction Prompts: The Schema Contract

An extraction prompt asks the model to find information that already exists in the provided text and return it in a specified structure. There is no generation, no inference, no creativity. The model is acting as a precise reader, not a writer.

Extraction prompts have three characteristics that distinguish them from conversational prompts:

1. The schema is mandatory. You must include the exact JSON schema in the prompt. Do not describe the output in prose ("return a JSON object with the person's skills"). Provide the literal schema with field names, types, and example values. The model fills in the blanks; it does not invent the structure.

2. Temperature should be near zero. Extraction is a deterministic task: the candidate's name is their name, not a creative interpretation of it. Setting `temperature=0.0` or `temperature=0.1` reduces the chance of the model paraphrasing, embellishing, or hallucinating a detail that isn't present.

3. The anti-hallucination constraint is explicit. Include a rule like: *"If a field is not present in the text, return an empty string or an empty array. Never invent information."* Without this, models will often fill in missing fields with plausible-but-wrong guesses.

§6.2 Evaluation Prompts: Grounding in a Rubric

An evaluation prompt asks the model to judge something against a set of criteria. This is where the analysis pipeline earns its value — and also where it is most likely to go wrong if you are not careful.

The key principle is rubric grounding: the criteria you want the model to use must be embedded in the prompt, verbatim, not left to the model's general opinion. An ungrounded evaluation prompt — *"Evaluate how well this resume fits this job"* — will use the model's training-time priors about what a good resume looks like. Those priors may be excellent or may be completely misaligned with the standards you actually care about.

A grounded evaluation prompt looks like this:

```
You are an ATS keyword auditor evaluating resumes against Singapore
entry-level technical job requirements.
```

```
EVALUATION CRITERIA (use these exactly — do not substitute your own):
  - Criterion 1: [exact criterion text]
  - Criterion 2: [exact criterion text]
  ...

Given the resume profile JSON and JD profile JSON below, evaluate
the resume against each criterion and return a JSON report.
```

By embedding the criteria, you ensure that two different models — GPT-4o, Claude 3.5, a local Llama model — would all evaluate against the same standard. The evaluation becomes auditable: you can point to which criterion produced which score.

> 💡
>
> **Why this matters for trust:**
>
> When the model gives a score of 65/100, the user's natural question is "why?" If the criteria are embedded in the prompt and the model cites them in its output, the answer is traceable. If the criteria came from the model's vague training-time sense of "good", the answer is "because it thinks so" — which is not useful feedback.

MULTIPLE CHOICE ch4-q7

An evaluation prompt produces a score of 78% for a resume. A colleague asks, "What criteria produced that score?" Which design choice makes this question answerable?

- ○ Using a higher model (e.g. GPT-4 instead of GPT-4o-mini) for more reliable reasoning.
- ○ Embedding the evaluation criteria verbatim in the prompt and requiring the model to cite each criterion in its JSON output.
- ○ Setting `temperature=0.0` so the model always produces the same score.

Check

§6.3 The Feedback-Only Principle

This section addresses a design decision that may seem counterintuitive at first: **why should an AI analysis tool tell you what's missing, but never rewrite your document to fix it?**

The answer has three layers.

Layer 1 — Accuracy. An AI that rewrites your resume is generating new content. Generated content may be fluent and plausible while being factually wrong about you — claiming skills you don't have, fabricating projects you didn't work on, or inflating titles. An analysis tool that only identifies gaps leaves the correction entirely to you, which ensures the result is accurate.

Layer 2 — Ownership. A resume, a portfolio, a proposal — these are artefacts of *your* professional identity. AI-generated language is generic by nature; it optimises for persuasiveness, not authenticity. Feedback that points to a gap empowers you to close it in your own voice. Feedback that closes it for you produces a document that sounds like everyone else's.

Layer 3 — Trust and accountability. In a hiring context (and many other professional contexts), submitting AI-generated content without disclosure may violate institutional policies or ethical norms. A tool that is strictly diagnostic — *it evaluates; it does not write* — cannot be misused in this way, because it produces no content to submit.

The practical consequence is this: every output field in an analysis pipeline's JSON schema should contain *observations about* the document, not *replacements for* the document. A field called `"missing_keywords"` is fine. A field called `"improved_summary"` is not.

> ⚠️
>
> **If you ever see an LLM output containing a rewritten resume bullet, cover letter paragraph, or any other professionally submitted content, treat it as a failure of the pipeline's design constraints — not a helpful bonus.**

SHORT ANSWER ch4-q8

Your analysis pipeline includes a prompt that asks: "Identify any weak resume bullets and suggest one improved version for each." Explain in two sentences why this crosses the feedback-only boundary and what a compliant alternative would look like.

Write your answer here...

Show Model Answer

§6.4 Temperature as a Precision Dial

Running the same prompt at various temperatures like 0.0, 0.7, and 1.5 will change output and can be used to create *conversational* diversity. In analysis pipelines, temperature has a different role: it controls the **precision–nuance trade-off**.

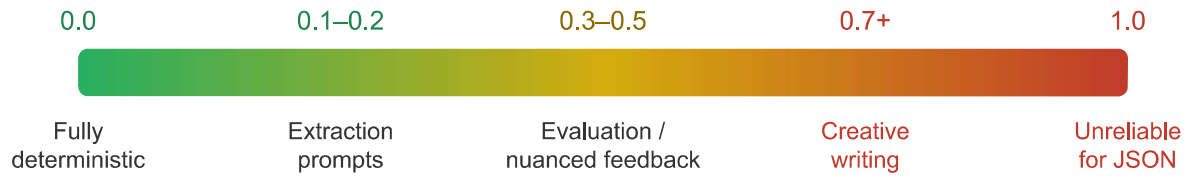

Figure 4.4: Temperature as a precision dial for analysis pipelines. Extraction stages benefit from near-zero temperature (exact copying of facts). Evaluation and feedback stages benefit from a small amount of variability to allow nuanced judgement. Values above 0.7 risk inconsistent JSON structure and should be avoided for any prompt that must return a parseable output.

The practical guidance is simple:

- **Extraction prompts:** `temperature=0.0` or `temperature=0.1` . The model is copying facts; randomness is a liability.
- **Evaluation/analysis prompts:** `temperature=0.2` to `temperature=0.3` . A small amount of variability allows the model to exercise judgment without destabilising the JSON structure.
- **Narrative feedback prompts:** `temperature=0.4` to `temperature=0.6` . These produce human-readable summaries where some stylistic variation is acceptable.
- **Above 0.7 for JSON-returning prompts:** a known source of malformed output. Avoid.

---

§7 Scoring: From Qualitative to Quantitative

§7.1 Why Numbers Build Trust

Qualitative feedback — *"Your resume is missing several important keywords"* — is useful but not actionable on its own. A candidate cannot tell whether they are close to passing or far away, or which gap matters most.

A numeric score — *"72 out of 100; you need 60 to pass the automated ATS filter"* — is immediately actionable. It tells the candidate how far they are from a threshold, which creates a concrete goal. It also makes the tool's outputs comparable across different sessions and different candidates.

This is not to say that numbers are always more trustworthy than prose. A poorly constructed score can give false precision — a number that *feels* authoritative but was computed from unreliable inputs. Good scoring design is transparent about its inputs and its formula.

§7.2 Weighted Aggregation

An analysis pipeline often produces several sub-scores — one for keyword match, one for document structure, one for bullet quality, and so on. These are combined into a single composite score via weighted aggregation.

The formula is:

$$S = \sum_{i=1}^{n} w_i \cdot s_i \tag{4.1}$$

where s_i is the sub-score for dimension i (on a 0–100 scale), w_i is the weight for that dimension, and all weights sum to 1:

$$\sum_{i=1}^{n} w_i = 1.0 \tag{4.2}$$

Why weights matter: not all dimensions contribute equally to the final decision. For ATS screening, keyword presence is the primary filter (high weight). Document structure matters less to an automated scanner than to a human reader (lower weight). Weights encode the *priorities of the evaluator* — which means they should be chosen deliberately and documented, not left to arbitrary defaults.

For example, if a system uses three dimensions with weights 0.5, 0.3, and 0.2:

Dimension	Weight (w_i)	Sub-score (s_i)	Contribution
Keyword match	0.5	80	40.0
Bullet quality	0.3	65	19.5
Structure audit	0.2	70	14.0
Total	**1.0**	—	**73.5**

The composite score from equation (4.1) is **73.5 out of 100**.

PREDICT OUTPUT ch4-q9

A scoring function uses these weights and sub-scores. What is the composite score (rounded to the nearest whole number)?

```
weights = {"keywords": 0.6, "structure": 0.25, "tone": 0.15}
scores  = {"keywords": 50,  "structure": 80,   "tone": 90}

composite = sum(weights[k] * scores[k] for k in weights)
print(round(composite))
```

Show Answer

§7.3 Thresholds and Their Meaning

A score of 64 out of 100 is only meaningful if you know what the threshold is. ATS pass thresholds — the minimum score a resume must achieve to pass automated screening — typically sit between 60% and 70% for most Singaporean technology roles. A composite score of 64 against a 60% threshold means the candidate passes; against a 70% threshold, they do not.

A well-designed analysis tool makes the threshold visible in its output. Rather than reporting only the score, it should report the score, the threshold, and whether the document passes or fails:

```
{
  "composite_score": 64,
  "pass_threshold": 60,
  "result": "PASS",
  "margin": 4
}
```

The `margin` field is especially useful: a margin of 4 means the candidate cleared the threshold comfortably; a margin of -8 means they need to add eight percentage points of keyword coverage before reapplying.

> ⚠️
>
> **The limits of thresholds:**
>
> ATS score thresholds are heuristics, not laws. Different companies set different thresholds. The same resume might pass the ATS for one company and fail for another. A good analysis tool communicates this uncertainty to the user rather than presenting the threshold as a universal standard.

---

§8 Handling Imperfect AI Output

§8.1 The LLM Is Not a Database

When you call `json.loads(response_text)`, you are making an assumption: that `response_text` is valid JSON. For a database query, that assumption is always safe. For an LLM response, it is frequently wrong.

LLMs are trained to produce *human-readable* text. JSON happens to be *machine-parseable* text that also looks human-readable. Models are quite good at producing JSON — but they are not perfect. Under the following conditions, a model may produce malformed JSON:

- **Leading prose:** The model adds *"Here is the JSON you requested:"* before the `{`.
- **Trailing prose:** The model adds *"I hope this helps!"* after the `}`.
- **Markdown fences:** The model wraps the JSON in triple backticks: ```` ```json ... ``` ````.
- **Wrong types:** A field defined as `["string"]` comes back as a single `"string"` (the model forgot the array).
- **Missing fields:** The model omits a field it considers obvious or empty.
- **Truncation:** The response hits `max_tokens` mid-object, producing invalid JSON.

Each of these has a known fix. Understanding them in advance prevents frustrating debugging sessions.

§8.2 Parsing Defensively

Defensive parsing means writing JSON-reading code that anticipates and handles the common failure modes rather than crashing on the first unexpected character.

A robust parsing function does three things:

1. **Strip markdown fences** before passing to `json.loads`.
2. **Find the first** `{` and parse from there, ignoring any leading prose.

3. **Catch** `json.JSONDecodeError` and surface an informative error (not a raw stack trace).

> **Listing 4.4** — A defensive JSON parser. Reference only.

```python
import json

def parse_llm_json(text: str) -> dict:
    """Parse JSON from an LLM response, tolerating common formatting issues."""
    # Step 1: strip markdown fences if present
    text = text.strip()
    if text.startswith("```"):
        end_of_first_line = text.find("\n")
        text = text[end_of_first_line + 1:]
    if text.endswith("```"):
        text = text[:text.rfind("```")]

    # Step 2: find the first '{' and parse from there
    start = text.find("{")
    if start == -1:
        raise ValueError("No JSON object found in LLM response.")

    try:
        obj, _ = json.JSONDecoder().raw_decode(text, start)
        return obj
    except json.JSONDecodeError as e:
        raise ValueError(f"LLM response is not valid JSON: {e}\n\nRaw text:\n{text}")
```

SPOT THE ERROR ch4-q10

This snippet tries to parse an LLM JSON response but has a bug. Which line causes the problem, and why?

```
1  raw = call_llm(prompt)          # returns a string
2  data = json.loads(raw)          # parse as JSON
3  skills = data["skills"]         # extract the field
4  print(skills)
```

Show Answer

§8.3 Type Coercion After Parsing

Even with a valid JSON object, fields may have unexpected types. A field defined as an array may come back as a string, or vice versa. Rather than crashing on a type mismatch, a pipeline can apply type coercion — nudging the value toward the expected type before passing it downstream.

> **Listing 4.5** — A minimal type coercion pattern. Reference only.

```python
def coerce_list_field(data: dict, field: str) -> dict:
    """
    Ensure `field` in `data` is a list.
    If the model returned a string instead, wrap it in a list.
    If the model returned None or an empty string, return an empty list.
    """
    value = data.get(field)
    if value is None or value == "":
        data[field] = []
    elif isinstance(value, str):
        data[field] = [value]   # wrap single string in a list
    # if it's already a list, leave it alone
    return data
```

§8.4 Retry and Fallback Patterns

In addition to plain retrying, analysis pipelines extend this with another scenario: JSON parse failure.

The clean pattern is to retry the LLM call when parsing fails, up to a maximum number of attempts. If all attempts fail, raise a descriptive error rather than returning `None` silently (which would propagate incorrect data through all downstream stages).

> **Listing 4.6** — Retry on JSON parse failure. Reference only.

```python
def ask_json_with_retry(system_prompt: str, user_message: str, max_attempts: int = 3) -> dict:
    last_error = None
    for attempt in range(max_attempts):
        try:
            raw = call_llm(system_prompt, user_message)
            return parse_llm_json(raw)
        except (ValueError, json.JSONDecodeError) as e:
            last_error = e
            if attempt < max_attempts - 1:
                print(f"Parse attempt {attempt + 1} failed: {e}. Retrying...")
    raise RuntimeError(
```

```
        f"Failed to parse LLM JSON after {max_attempts} attempts. "
        f"Last error: {last_error}"
    )
```

Notice the pattern: try → parse → catch parse error → retry → raise after all attempts exhausted. The function never returns `None` and never swallows an error silently.

MULTIPLE CHOICE ch4-q11

A retry function for LLM JSON parsing uses `return None` instead of `raise RuntimeError(...)` when all attempts fail. What is the most likely consequence downstream?

- ○ The program crashes immediately with a helpful error message.
- ○ Downstream stages silently receive `None` as their input, producing incorrect outputs without raising an error.
- ○ The LLM call is automatically retried once more by the Python runtime.

Check

---

§9 Worked Example: The Proposal Analyser

This section walks through a complete document analysis scenario from beginning to end. This is a *small* problem designed to make the pipeline pattern concrete.

The scenario: A student at a design school submits a project proposal. The school's marking panel has a fixed rubric with five criteria. We want to build a tool that: 1. Extracts the key claims from the proposal. 2. Evaluates each claim against the rubric. 3. Produces a score and an actionable feedback list.

Why this is a pipeline problem: A single prompt cannot simultaneously extract claims *and* evaluate them against five criteria *and* produce a score *and* generate feedback. Each subtask needs its own prompt.

§9.1 Stage 1 — Parse

The proposal arrives as a PDF. We call a text extractor (no LLM) and validate the output:

```
raw_text = extract_text("proposal.pdf")    # pypdf
validated = validate_extracted_text(raw_text, source="proposal")
# validated is now a str of 800–3000 characters — a reasonable proposal length
```

The extraction takes a few milliseconds. No API call, no cost.

§9.2 Stage 2 — Extract the Proposal Profile

We send the text to the LLM with an extraction prompt. The schema we designed in advance:

```
{
  "project_title":    "string",
  "problem_statement": "string",
  "proposed_solution": "string",
  "target_users":     ["string"],
  "deliverables":     ["string"],
  "timeline_weeks":   "integer or null"
}
```

System prompt (**ICCO**):

```
Instruction: Extract structured information from the project proposal below.
Context: This is a student proposal from a design school. The reader is an automated evaluatio
Constraints: Only extract what is explicitly stated. If a field is absent, return "" or []. Ne
Output: Return ONLY a valid JSON object matching this schema: { "project_title": "...", ... }
```

The model returns a JSON object. We parse it with `parse_llm_json()` and validate the types.

`temperature=0.1` .

What we know at this point: We have a clean, machine-readable summary of what the student claims to be delivering.

§9.3 Stage 3 — Evaluate Against the Rubric

We send *both* the extracted proposal profile *and* the rubric to a second LLM call. The rubric is embedded verbatim in the system prompt — rubric grounding in practice:

```
You are an academic reviewer evaluating project proposals.

EVALUATION CRITERIA (apply these exactly):
  1. Problem clarity: Is the problem clearly defined? (score 0–20)
```

```
    2. Solution feasibility: Is the solution achievable in the stated timeline? (score 0-20)
    3. User specificity: Are target users described with meaningful detail? (score 0-20)
    4. Deliverable concreteness: Are deliverables specific and measurable? (score 0-20)
    5. Originality: Does the solution go beyond obvious or trivial approaches? (score 0-20)

  PROPOSAL PROFILE:
  [insert profile JSON here]

  Return a JSON object with a score and a one-sentence observation for each criterion.
```

`temperature=0.2` . The model scores each criterion from 0–20.

§9.4 Stage 4 — Score and Report

No LLM needed here. Pure Python aggregates the sub-scores using equation (4.1). In this equal-weighted case:

$$S = \frac{1}{5}\sum_{i=1}^{5} s_i = \frac{14 + 18 + 10 + 16 + 12}{5} = \frac{70}{5} = 14 \text{ (on a 0–20 scale)}$$

Normalised to 0–100: $14/20 \times 100 = 70$. Pass threshold is 60. The proposal passes.

The report is assembled from the criterion observations and the composite score. The output is:


```
{
  "composite_score": 70,
  "pass_threshold": 60,
  "result": "PASS",
  "criteria": {
    "problem_clarity":      {"score": 14, "note": "Problem is stated but lacks quantification.
    "solution_feasibility": {"score": 18, "note": "Timeline is realistic for the stated delive
    "user_specificity":     {"score": 10, "note": "Target users described only as 'students' –
    "deliverable_concrete": {"score": 16, "note": "Three of four deliverables are specific and
    "originality":          {"score": 12, "note": "Solution is competent but builds on an esta
  }
}
```


Key observation: Every piece of feedback is an observation about the proposal's content, not a rewrite of it. "Target users described only as 'students' — too broad" tells the student exactly what to fix. It does not rewrite their target-user section for them.

§9.5 Reflection Questions

Before moving on, consider these questions. They do not have single correct answers; they are prompts for engineering thinking.

1. If Stage 3 produced `"score": "fourteen"` (a string) instead of `"score": 14` (an integer), which part of the pipeline should catch this, and how?
2. The proposal is 4,000 words. The model's context window is 16,000 tokens. Will the proposal fit in a single call? How would you estimate this? (Keep in mind that there are approximately 4 characters per token.)
3. If the rubric criteria change next semester (new grading standards), which stage(s) need to change? Which stay the same?
4. The `user_specificity` sub-score is 10/20 — the weakest criterion. But the student scored 18/20 on feasibility. Is it helpful or harmful to show only the composite score (70) without the breakdown? Why?

---

§10 Common Pitfalls

About this section

These pitfalls come from real AI engineering practice. Each one looks obvious in hindsight — but most developers encounter at least three of them on a first pipeline project.

Pitfall 1 — Asking one prompt to do everything. A single prompt that tries to extract, evaluate, score, and generate feedback produces worse results in each dimension than four specialised prompts. Symptoms: inconsistent JSON structure, missing fields, narrative mixed with data. Fix: one prompt, one responsibility.

Pitfall 2 — Not validating text extraction. Passing an empty string (from an image PDF) or a 150,000-character string (from an unexpectedly long document) to an LLM will either produce a hallucinated extraction or fail with a context-length error. Fix: validate character length before any LLM call.

Pitfall 3 — Using bare `json.loads()` on LLM output. LLMs frequently add polite preamble, markdown fences, or trailing commentary. A bare `json.loads()` call will fail on any of these. Fix: strip fences, find the first `{`, use `JSONDecoder.raw_decode()`.

Pitfall 4 — Trusting the model's own score. Asking the model to produce a score ("give this a score out of 100") and then using that number directly invites drift. The model's intuitive scoring may not match your weighted

formula. Fix: compute the composite score in Python from the sub-scores the model produced; treat the model's direct score estimate as a cross-check, not the ground truth.

Pitfall 5 — High temperature on extraction prompts. At `temperature=0.7`, a model may creatively embellish a skill list, adding technologies it inferred the candidate "probably knows" from context. This is hallucination — not a model failure, but a prompt-design failure. Fix: use `temperature=0.0 – 0.1` for any prompt that must extract verbatim content.

Pitfall 6 — Schema drift between stages. Renaming a field in Stage 2's output without updating Stage 3's input expectation produces a silent bug — `result.get("old_field_name", [])` returns `[]` without raising an error. Fix: define schemas as Python constants at the top of each module; search for all usages before renaming.

Pitfall 7 — Swallowing parse failures. Returning `None` or `{}` on a JSON parse error and continuing the pipeline is the most dangerous pattern. Downstream stages receive corrupt data and produce incorrect outputs with no indication that anything went wrong. Fix: raise an exception; let the caller decide whether to retry.

Pitfall 8 — Prompting for content generation inside a feedback-only tool. A prompt that says "suggest improvements" or "rewrite the weak bullet" crosses the feedback boundary. Fix: ask the model to *identify* what is wrong and *which criterion* it violates — not what the correct version would look like.

---

§10.5 Chapter Self-Check

These questions cover all seven Learning Outcomes. Work through them to validate your understanding.

FILL IN THE BLANK ch4-sc1

Data stored as free-form natural language, where the location of any specific fact is unpredictable to a computer, is called ... data.

Check

TRACE THE FLOW ch4-sc2

Arrange these four pipeline activities in the correct order for a document analysis system.

Compute a weighted composite score in Python from sub-score integers

Run a text extractor library on the uploaded file to get a plain string

Send the structured document profile and requirements profile to the LLM for comparison

Send the plain text to the LLM with a schema-shaped extraction prompt

Check Order

MULTIPLE CHOICE ch4-sc3

You are designing a Stage 2 extraction prompt for a new pipeline. Which of the following is the most important thing to decide BEFORE writing a single word of the prompt?

- ○ Which LLM provider to use (OpenAI vs Anthropic vs Ollama).
- ○ The exact JSON schema that Stage 3 expects to receive.
- ○ The temperature value to use for the extraction.

Check

MULTIPLE CHOICE ch4-sc4

An evaluation prompt says: "Score this resume from 0 to 100 based on how strong it looks." No criteria are embedded. What is the main problem with this prompt?

- ○ The model cannot produce a number without explicit instructions.
- ○ The score is based on the model's general opinion, not a defined set of criteria — making it unexplainable and inconsistent across different models.
- ○ The model will refuse to produce a score without more context.

Check

MATCH CONCEPTS ch4-sc5

Match each pipeline prompt type to its recommended temperature range.

Extraction prompt (copy facts from text into JSON) -- select --

Evaluation prompt (score document against rubric) -- select --

Narrative feedback prompt (summarise findings for a human) -- select --

Any JSON-returning prompt -- select --

Check

SPOT THE ERROR ch4-sc6

This retry function silently swallows parse failures. Identify the line that causes silent failure and explain what it should do instead.

```
1  def ask_json(prompt, user):
2      for attempt in range(3):
3          try:
4              raw = call_llm(prompt, user)
5              return json.loads(raw)
6          except json.JSONDecodeError:
7              pass
8      return {}
```

Show Answer

SHORT ANSWER ch4-sc7

A colleague proposes adding a field called `"improved_bullets"` to the analysis output schema, each entry being a rewritten version of a weak resume bullet. Write two sentences: one explaining the problem with this approach, and one describing what a feedback-only alternative field would look like.

Write your answer here...

Show Model Answer

§11 Summary

This chapter introduced the multi-stage AI pipeline as the foundational pattern for document analysis systems. Where a single chatbot call produces one undifferentiated response, a pipeline breaks the work into independent, testable stages — each with a single responsibility, a defined input contract, and a defined output schema.

We explored four major concepts:

1. **Data shapes.** Documents start as unstructured text. Pipelines transform text into structured JSON objects, which code can interrogate, compare, and store. The LLM is the bridge; JSON is the contract.
2. **Multi-stage design.** Extraction, evaluation, scoring, and reporting are different tasks that require different prompts, different temperatures, and different output formats. Combining them in a single call reduces quality across all dimensions.
3. **Prompt engineering for analysis.** Extraction prompts need low temperature, anti-hallucination constraints, and an explicit schema. Evaluation prompts need rubric grounding — criteria embedded verbatim — to be auditable and trustworthy. All analysis prompts adhere to the feedback-only principle: describe what's wrong; never rewrite.
4. **Defensive output handling.** LLM-produced JSON is not guaranteed to be clean. A production pipeline strips markdown fences, finds the first `{`, handles type mismatches with coercion, and raises explicit errors rather than returning `None` or `{}` silently.

Key facts: - Stage independence means each stage can be tested without the others existing. - Schema-first design means writing the JSON schema before writing the prompt. - The weighted composite score formula is $S = \sum_i w_i \cdot s_i$ where weights sum to 1. - Temperature for extraction: 0.0–0.1. For evaluation: 0.1–0.3. Above 0.7 risks JSON malformation. - Feedback-only pipelines describe gaps; they never generate replacement content.

Quick Reference

Key terms throughout this chapter include inline definitions — hover over (or tap on mobile) any underlined term to see its explanation. The table below collects them in one place for quick lookup.

Term	Definition
ATS (Applicant Tracking System)	Automated software used by employers to filter job applications by keyword and criteria matching before a human recruiter reviews them.
composite score	A single numeric score produced by aggregating multiple sub-scores using a weighted average formula.
defensive parsing	A JSON-reading strategy that anticipates and handles common LLM output issues — markdown fences, leading prose, type mismatches — rather than assuming the output is clean.
evaluation pipeline	A multi-stage AI system that takes a document, extracts structured information from it, and evaluates that information against predefined criteria to produce a score and feedback.
extraction pipeline	The subset of a document analysis pipeline responsible for converting unstructured text into a typed JSON object using a schema-shaped LLM prompt.
extraction prompt	An LLM prompt whose primary task is to copy facts from provided text into a specified JSON schema, without generating, paraphrasing, or inferring content that is not present.
evaluation prompt	An LLM prompt whose primary task is to compare two structured inputs against a set of criteria and produce a scored analysis.
feedback-only design	A pipeline design principle that constrains AI outputs to observations, scores, and identified gaps — never to generated replacement content such as rewritten text.
rubric grounding	The practice of embedding evaluation criteria verbatim in an evaluation prompt so the model judges against defined standards rather than training-time priors.
schema-first design	The discipline of specifying the exact JSON output schema before writing the prompt that produces it, ensuring that downstream stages can depend on a stable field contract.
schema contract	The implicit or explicit agreement between two pipeline stages about the exact field names, types, and structure of the data passed between them.
stage independence	The property of a pipeline stage that allows it to be understood, tested, and replaced without knowledge of the stages before or after it.
structured data	Data stored in a format with predictable field names, types, and locations — such as a JSON object or a database row — that code can interrogate directly.

Term	Definition
text extraction	The process of recovering human-readable text from a binary file format (such as PDF) by reading the file's internal drawing or encoding instructions.
type coercion	The process of converting a value to its expected type — for example, wrapping a string in a list when an array is expected — to tolerate common LLM type mismatches without crashing.
unstructured data	Data encoded as natural language prose, where the location of any specific fact is not predictable to a program without reading and understanding the full text.
weighted aggregation	The process of combining multiple sub-scores into a single composite score by multiplying each sub-score by its assigned weight and summing the results.